

Question 1.

Answer the following questions briefly. Give reason(s) to support your answer where necessary. 15 points

- (a) Suppose we want to search an element k in a Max-Heap. What is the cost of searching an element in Max-Heap? (2)

✓ Cost of search in Max-Heap is $O(n)$

Max-Heap will sort the array in decreasing order such that maximum number is on root and ~~a rest~~ all node i are less than equal to their parent nodes.

We just have to iterate the max heap to find the desired output. It will cost $O(n)$.

- (b) Is $a^{n+1} = O(a^n)$? (here a is a constant) (2)

$$a^{n+1} = O(a^n)$$

$$✓ a^n \cdot a^1 = O(a^n)$$

as a is constant

$$c a^n = O(a^n)$$

$$O(a)^n = O(a^n) \quad \text{Hence proved}$$

left hand side = R.H.S

- (c) Is $\log_3 n = O(\log_2 n)$? (2)

L.H.S

$$\log_3 n$$

As we know that:

$$\log_b a = \frac{\log_c a}{\log_c b}$$

By using this formula

$$\log_3 n = \frac{\log_2 n}{\log_2 3}$$

where

$$\frac{1}{\log_2 3} = \text{constant so ignore it.}$$

$$\log_3 n = \log_2 n$$

$$= O(\log_2 n)$$

R.H.S

$$= O(\log_2 n)$$

Hence proved

$$L.H.S = R.H.S$$

2. Write a recurrence relation that satisfies all the conditions of case-3 of Master Method but does not satisfy the regularity condition. Show your working also.

First we will calculate $4T\left(\frac{n}{2}\right) + n^2 \log n$ here $a=4, b=2$
 $f(n) = n^2 \log n$
 $= \frac{n^{\log_a b}}{\log_a a}$
 $= \frac{n^{\log_2 4}}{\log_2 2}$
 $\therefore \log_2 4 = 2$

Now compare n^2 it with $f(n)$ it satisfies the first condition of Master method case 3.
 now check regularity condition $\frac{f(n)}{n^2} \leq \frac{f(n)}{n^2 \log n}$

$$4T\left(\frac{n}{2}\right) \leq cn^2 \log n \Rightarrow 2n^2 \log n \leq cn^2 \log n \Rightarrow 2 \leq c \rightarrow c \text{ should be greater than 1}$$

- (e) If I prove that an algorithm takes $\Theta(n^2)$ worst-case time, is it possible that it takes $O(n)$ on some inputs?

Yes it is possible as n^2 is the lower bound of the algorithm which tells us that the algorithm can never go below that lower bound. Suppose that lower bound is $\Omega(n)$ then it is possible that it takes $O(n)$ on some inputs.

- (f) If I prove that an algorithm takes $\Theta(n^2)$ worst-case time, is it possible that it takes $O(n)$ on all inputs?

No it is not possible as the worst case time complexity is $O(n^2)$ which is the upper bound of the algo.

For all inputs it must reach the upper bound which is $O(n^2)$

Question 2 15 points

Suppose Ahmad suggests a divide and conquer algorithm for solving some problem. Considering the input size n , Ahmad's algorithm divides the problem into 8 sub-problems each of size $\frac{n}{2}$. Combining the results of the 8 sub problems cost n^2 .

Sara came up with another divide and conquer algorithm for solving the same problem. Sara's algorithm divides the problem into 2 sub-problems of different sizes; 1st sub-problem of size $\frac{n}{8}$ and 2nd sub-problem of size $\frac{7n}{8}$. Combining the results of the 2 sub problems costs n .

Assuming that both the algorithms take constant time if $n = 1$, then find out whose algorithm is the best asymptotically? Show your working.

Recurrence eq for algo 1

$$= 8T\left(\frac{n}{2}\right) + n^2$$

Recurrence eq for algo 2

$$= T\left(\frac{n}{8}\right) + T\left(\frac{7n}{8}\right) + n$$

For algo 1

Using Master Method

First step: find $n^{\log_b a}$

Second step: compare it with $f(n)$

Third step: check which case is implementing

$$8T\left(\frac{n}{2}\right) + n^2$$

Here

$$a = 8, \quad b = 2, \quad f(n) = n^2$$

$$n^{\log_b a} = n^{\log_2 8} \quad \log_2 8 = 3$$

$$= n^3$$

Now by comparing

$$n^3 \geq n^2$$

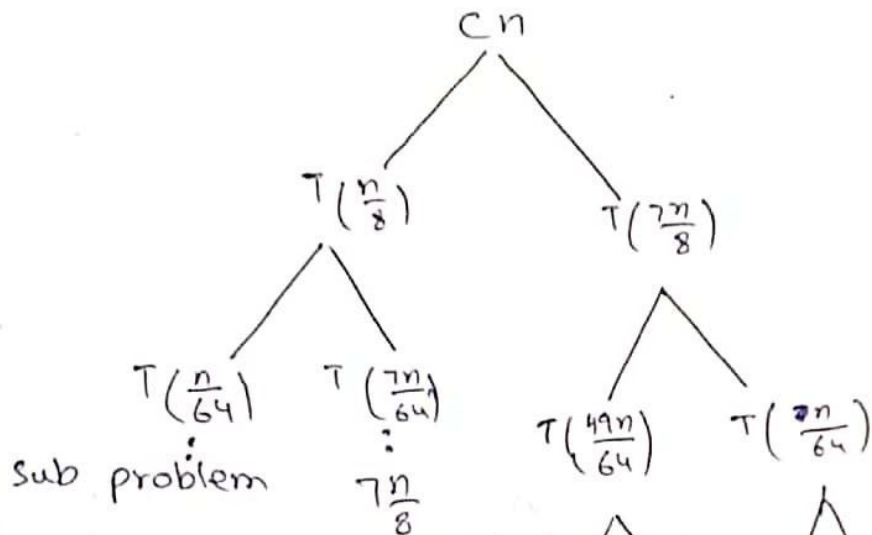
$\therefore$ greater than $f(n)$

This page is intentionally left blank

$$O(n^3), O(n^3)$$

For algo 2 we will use recurrence tree to find its complexity.

$$T\left(\frac{n}{8}\right) + T\left(\frac{7n}{8}\right) + n$$



Sub problem $\frac{7n}{8}$ will have more effect on the tree so will ignore $T\left(\frac{n}{8}\right)$ and go on with $T\left(\frac{7n}{8}\right)$ to find the total levels of the tree

$$\begin{aligned} \text{when } n &= 1 \\ \left(\frac{7}{8}\right)^2 n^2 &= 1 \\ n^2 &= \left(\frac{8}{7}\right)^2 \end{aligned}$$

Apply log on both sides

$$\log n = \log \left(\frac{8}{7}\right)^2 = \log_{\frac{8}{7}} n$$

$$T(n) = \text{levels} + cn$$

Conclusion :

Hence proved algo 2 Sara's algo. is asymptotically best.

Question 3.....

15 points

Describe an algorithm that, given n integers in the range 0 to k , preprocesses its input and then answers any query about how many of the n integers fall into a range $[a \dots b]$ in $O(1)$ time. Your algorithm should use $O(n + k)$ preprocessing time.

Here we will use counting sort algo as this algo provides answers of any query (range $a \dots b$) in constant time

Counting-sort (A, B, k)

create an array of size equal to k

1. $C[k]$

Initialize The whole array with zeros

2. for i is 1 to k

3. $C[i] = 0$

For loop for storing frequency of each number from a given array in Array C

4. for i 1 to n

5. $C[A[i]] = C[A[i]] + 1$

Commulative sum

6. for i 1 to n

7. $C[A[i]] = C[A[i]] + C[A[i] - 1]$

8. for $j = A.length$ down to 1

9. $B[C[A[j]]] = A[j]$

10. $C[A[j]] = C[A[j]] - 1$

Time Complexity = $O(n + k)$

Question 4 15 points

Consider two sorted arrays A and B containing real numbers. Both A and B has $\frac{n}{2}$ elements each. Both A and B may contain duplicate entries. Define the sorted-union $C = A \cup B$ by making the sorted array C out of A and B with no duplicate entries. Write an $O(n)$ time algorithm to find the sorted-union. You can use as much extra space as you like.

	1	2	3	4	5	6
A	4	5	6	7	8	9

	1	2	3	4	5	6
B	2	3	9	11	12	13

size of both array = $\frac{n}{2}$

1. m_1 size of A = $\frac{n}{2}$

2. m_2 size of B = $\frac{n}{2}$

3. $i = j = 1$

} $O(1)$

4. $B[m_2+1] = A[m_1+1] = +\infty$

create a new array C of size $m_1 + m_2 + 1$

5. ~~create C[n]~~

6. ~~create C[n]~~

7. for k = 1 to n

8. ~~if (A[i] != C[k] && B[j] != C[k])~~

9. if (A[i] < B[j] && A[i] != C[k])

10. C[k] = A[i]

11. i = i + 1

12. else if (B[j] < A[i] && B[j] != C[k])

13. C[k] = B[j]

14. j = j + 1

} $O(n)$

Time complexity of this algo is $O(n)$ which equal to $O(m_1 + m_2)$

Question 5 15 points
 You are given an array $A[1 \dots n]$ of n sorted distinct integers (not necessarily positive) and two integers i and j satisfying $1 \leq i \leq j \leq n$. We want to find an index k such that $i \leq k \leq j$ and $A[k] = k$, provided such an index exists.

- (a) Assume $n = 9$ and consider an arbitrary sorted list $A[1 \dots 9]$. Suppose you only know the value of $A[4]$, and that $A[4] \neq 4$. For at least how many integers k between 1 and 9 (excluding 4) can you say that $A[k] \neq k$?

Given that: $A[4] \neq 4$

1	2	3	4	5	6	7	8	9
			*					

As stated in the question statement that we are given an arbitrary subarray somewhere in the array

if $A[4] \neq 4$

It means that indexes after 4 must be in form of sorted array till 9

1	2	3	4	5	6	7	8	9
?	?	?	$\neq$	5	6	7	8	9

So, There must be 3 integers between 1 and 9 (excluding 4) where

$$A[k] \neq k.$$

- (b) Give a Divide-and-Conquer algorithm that can solve the above problem (for a general value of n). To receive full credit, the runtime of your algorithm on input $(A, 1, n)$ must be in $O(\lg n)$. (10)

7 Here we can use partition algo as it given time complexity $O(\lg n)$

Partition($A, 1, n$)

Pivot = $A[n]$

$i = 1$

for $j = 1$ to $n-1$

if $A[j] < \text{Pivot}$

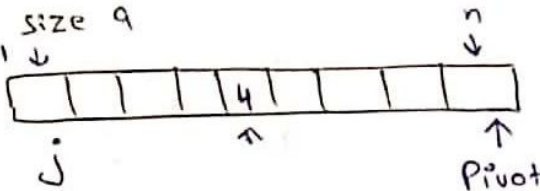
~~$i = i + 1$~~

exchange ($A[j] \leftrightarrow A[i]$)

exchange (~~A~~ Pivot, $A[i+1]$)

return $i + 1$

Suppose an array of size n at starting $\leftarrow i = 0$



$\times$ Find_Integer($A, 1, n$)

$s = 1$

$e = A.length$

if ($s > e$)

return -1

$mid = \lfloor \frac{s+e}{2} \rfloor$

if ($A[mid] < A[mid+1]$ & $A[mid] > A[mid-1]$) return mid

if ($A[mid] > A[mid+1]$)

call recursion Find_Integer(A, mid, n)

else

call recursion Find_Integer(

(g) What is the height of the recursion tree formed by:

$$T(n) = \begin{cases} aT(\frac{n}{b}) + f(n) & \text{if } n > 1; \\ 1 & \text{if } n = 1. \end{cases}$$
$$a \geq 1, b > 1$$

2

$$= \log_b n + 1$$

Question 6 15 points

An $n \times n$ matrix is monotone if each of its rows and columns are sorted in nondecreasing order. For example,

$$A = \begin{matrix} & \begin{matrix} 1 & \dots & N \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ N \end{matrix} & \begin{pmatrix} 1 & 12 & 17 & 19 \\ 15 & 18 & 20 & 25 \\ 16 & 23 & 26 & 30 \\ 24 & 27 & 31 & 35 \end{pmatrix} \end{matrix}$$

is monotone. Given monotone matrix A and value a , the membership problem is to test whether $a \in A$ or $a \notin A$. More specifically $MEM(A, a, x1, y1, x2, y2)$ will return *TRUE* if there exists i, j such that $a = A[i, j]$ where $x1 \leq i \leq x2$ and $y1 \leq j \leq y2$ and *FALSE* otherwise. (Note that we are labeling the rows so that the top row is 1 and the bottom one N , while the left column is 1 and the right column is N .) For example, in the given matrix, $MEM(A, 31, 1, 1, 4, 4)$ is *TRUE* while $MEM(A, 31, 1, 1, 3, 3)$ is *FALSE*.

A simple way to solve this problem is to test each element of A for equality with a and this can be achieved in $O(n^2)$. You are required to design an $O(n)$ time algorithm to solve the membership problem.

~~We~~ with

As the given problem is in sorted form the rows and columns are sorted in decreasing order we just call ~~for~~ for loop to find the desired output as the complexity of for loop is $O(n)$.